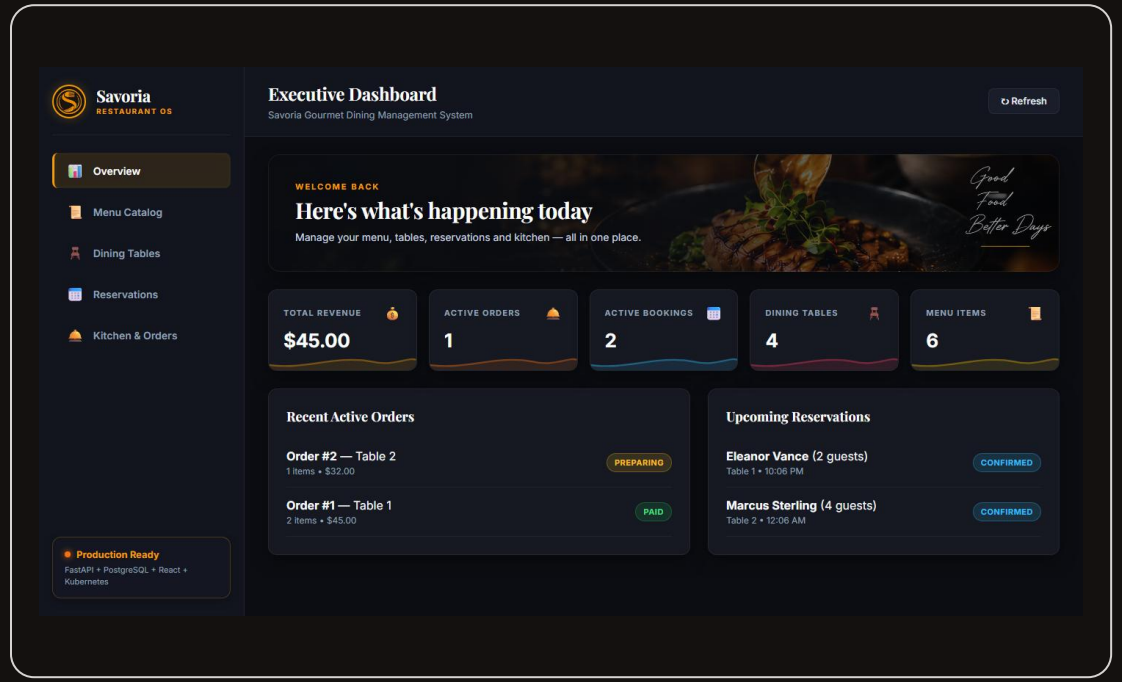
DEVOPS ENGINEERING CAPSTONE · SESSION 21

Savoria

Restaurant Management Platform

A cloud-native platform for menu catalog, dining floor, reservations and kitchen orders, delivered through the complete 10-module DevOps lifecycle.



Evidence: Savoria executive dashboard (React + Vite)

Author: M S Sayed (@MSSAYED11) | DevOps Engineering Capstone

M1 MODULE 1 · APPLICATION CORE

FastAPI Backend, PostgreSQL & React Dashboard

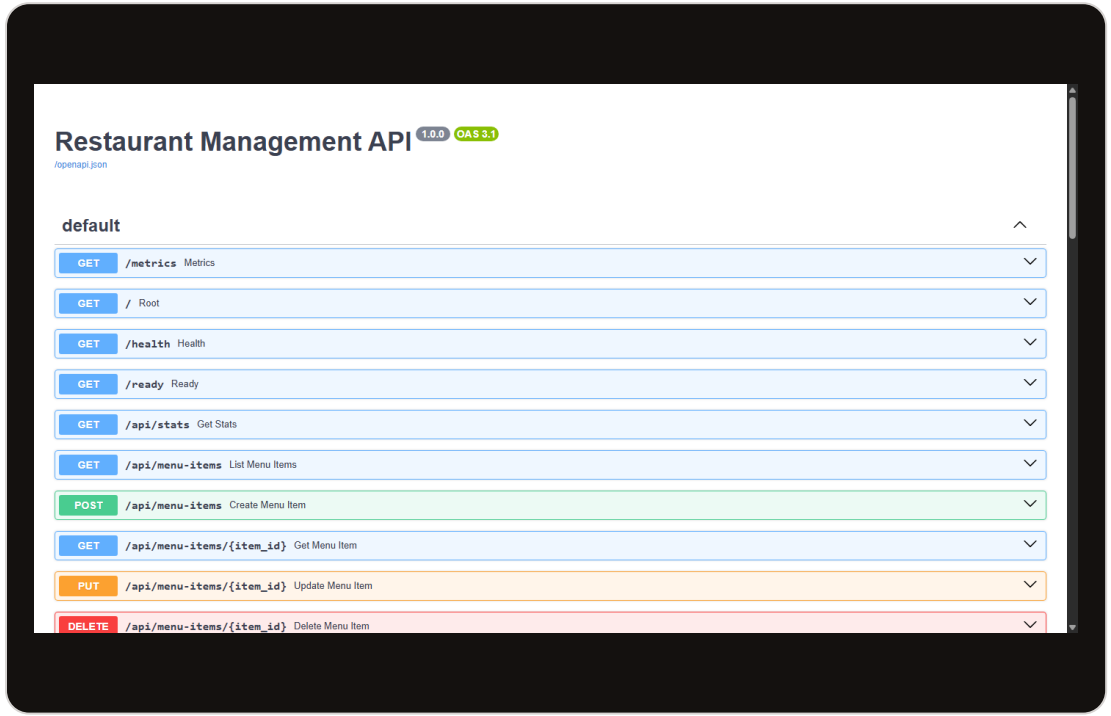
Fulfilling M1 criteria: /health probe, 4+ REST endpoints, Alembic migrations and a React UI

1 **Probes and 21 REST routes**
/health, /ready and /metrics, plus full CRUD under /api for menu, tables, reservations and orders.

2 **PostgreSQL with Alembic**
PostgreSQL 16 schema versioned by Alembic; migrations run automatically when the backend starts.

3 **Business rules in the API**
Over-capacity bookings return 400, double-booking returns 409, orders follow PLACED to PAID.

4 **Dark-luxury React UI**
Vite/React app with live KPI cards, menu catalog, dining tables, reservations and kitchen orders.



Evidence: FastAPI Swagger UI at :8000/docs

Automated Pytest Suite with an Isolated Database

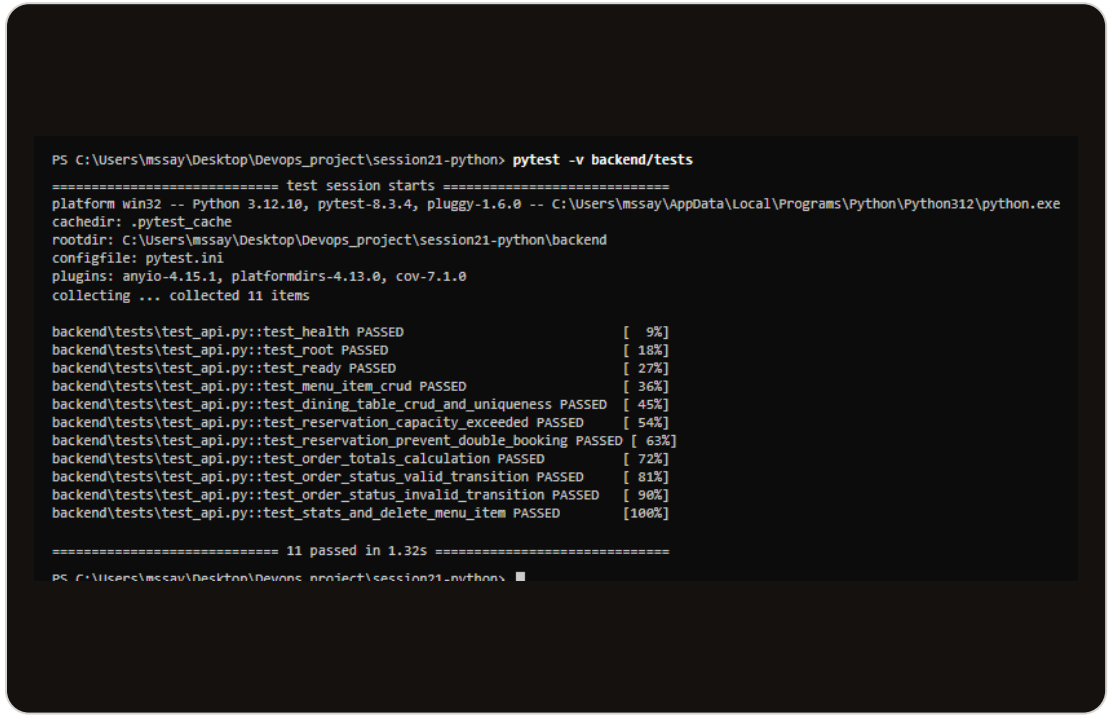
Fulfilling M2 criteria: clean pytest -v run, 5+ test cases, isolated database and pytest.ini

1. **11 test cases, all passing**
   Well past the 5-test minimum: health, ready, CRUD, capacity, double-booking and order tests.

2. **Business-rule coverage**
   Tests cover capacity limits, double-booking prevention and valid versus invalid order transitions.

3. **Isolated SQLite via conftest.py**
   SQLAlchemy dependency overrides keep every test off the PostgreSQL data.

4. **Fast regression gate**
   11 passed in 1.32 s; the same suite is the first CI job and blocks image builds if it fails.



Evidence: pytest -v, 11 of 11 tests passing (1.32 s)

Conventional Commits & Repository Hygiene

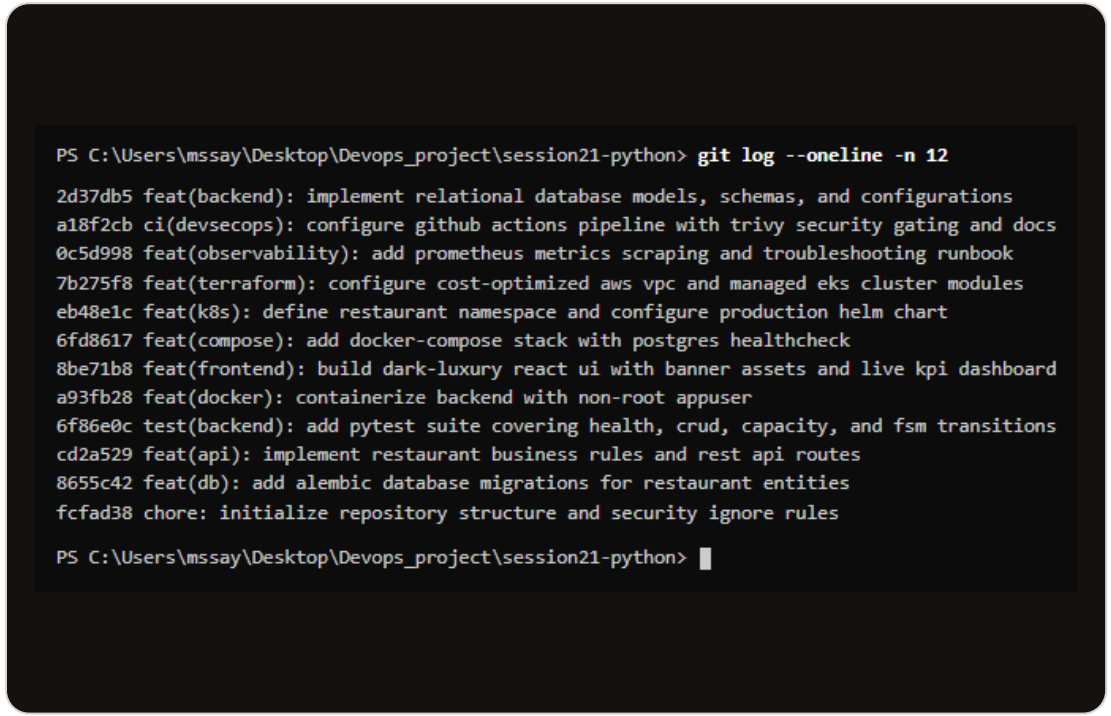
Fulfilling M3 criteria: 10+ meaningful commits, descriptive messages and a clean .gitignore

1. **24 project commits**
   22 follow Conventional Commits with feat, ci, test, fix, docs and chore prefixes and scopes.

2. **Scoped, meaningful messages**
   For example feat(db) for migrations, test(backend) for the pytest suite, feat(k8s) for the Helm chart.

3. **Strict .gitignore**
   Excludes .env, keys, __pycache__, .venv, node_modules, Terraform state and tfvars files.

4. **Secrets-free, modular repo**
   No credentials committed; backend, frontend, helm, terraform, k8s, monitoring and troubleshooting folders.



Evidence: git log --oneline -n 12

Non-Root Containers & a One-Command Compose Stack

Fulfilling M4 criteria: backend Dockerfile, multi-stage frontend, non-root users and Docker Compose

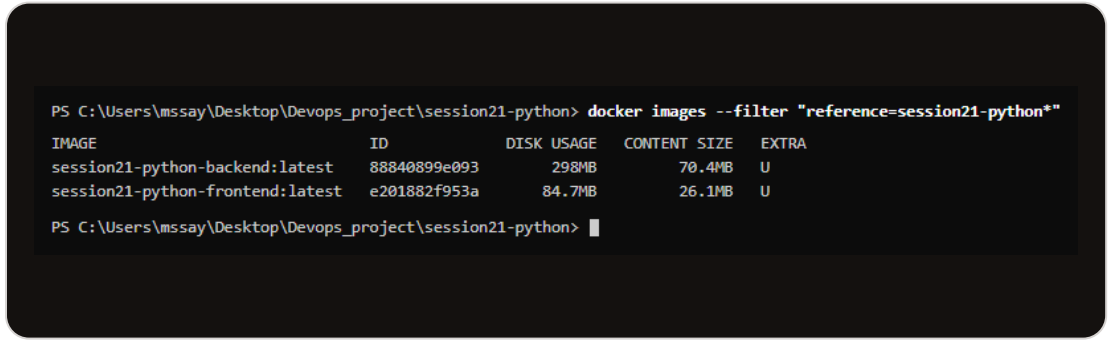
1. **Multi-stage frontend build**
   Node 22 Alpine builds the Vite bundle; only static files are copied into nginx:1.27-alpine.

2. **Non-root execution**
   Backend runs as appuser (UID 10001); the frontend runs as the nginx user on port 8080.

3. **Single-command Compose**
   docker compose up --build starts backend, frontend and PostgreSQL 16 with a database healthcheck.

4. **Lean images**
   Content size is 70.4 MB for the backend image and 26.1 MB for the frontend image.





Evidence: docker compose ps and docker images (full size in README)

3-Job GitHub Actions Pipeline Publishing to GHCR

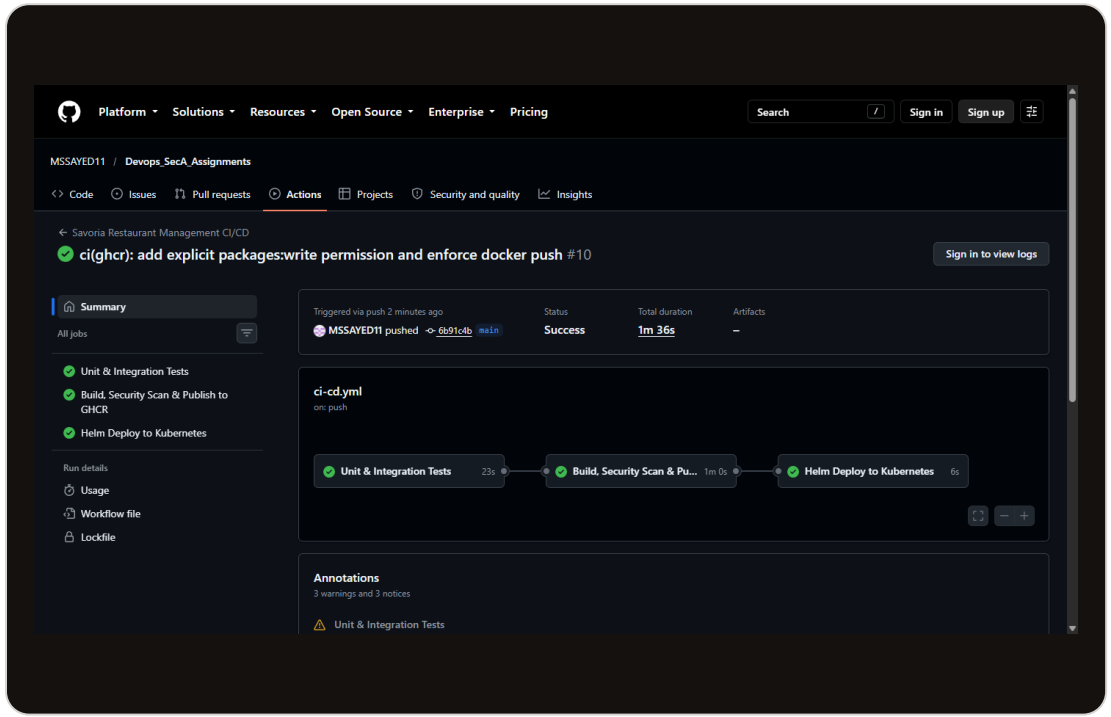
Fulfilling M5 criteria: push-to-main trigger, pytest gate, frontend build and SHA-tagged GHCR images

1. **Job 1: Tests (22 s)**
   Runs backend pytest and the Vite frontend build on every push and pull request to main.

2. **Job 2: Build, Scan, Publish (1 m)**
   Builds both images, runs Trivy, then pushes to ghcr.io/mssayed11.

3. **Commit-SHA image tags**
   Each image is pushed tagged with the full Git commit SHA, alongside a latest convenience tag.

4. **Job 3: Helm deploy (6 s)**
   Runs helm upgrade --install with the SHA tag; skips safely if no cluster credentials are set.



Evidence: green pipeline run #10, 1 m 36 s total

Trivy Container Vulnerability Scanning in CI

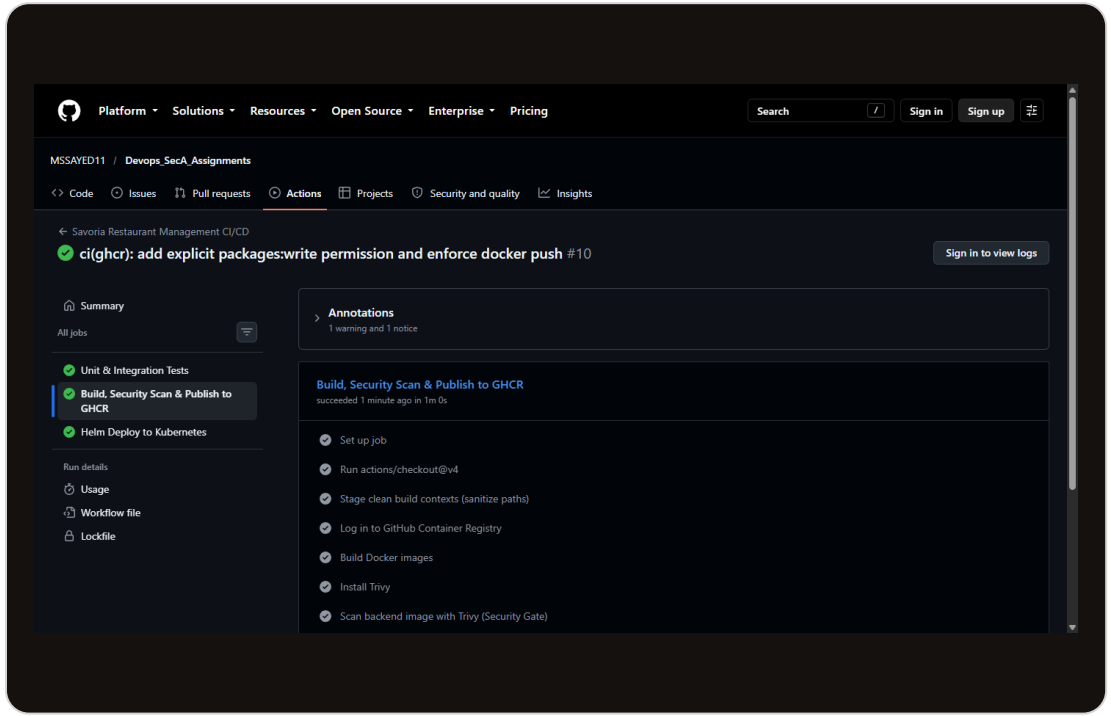
Fulfilling M6 criteria: Trivy scan on both images in CI, HIGH/CRITICAL severities and a written explanation

1. **Both images scanned**
   Backend and frontend images are scanned with Trivy inside the CI job.

2. **Scan runs before the push**
   The scan steps sit between the image build and the docker push to GHCR.

3. **HIGH and CRITICAL focus**
   Flags --severity HIGH,CRITICAL --ignore-unfixed report fixable serious CVEs only.

4. **Written explanation**
   trivy-explanation.md explains what is scanned and how to read the result.



Evidence: Trivy scan steps in the CI job log

AWS VPC & Managed EKS Defined in Terraform

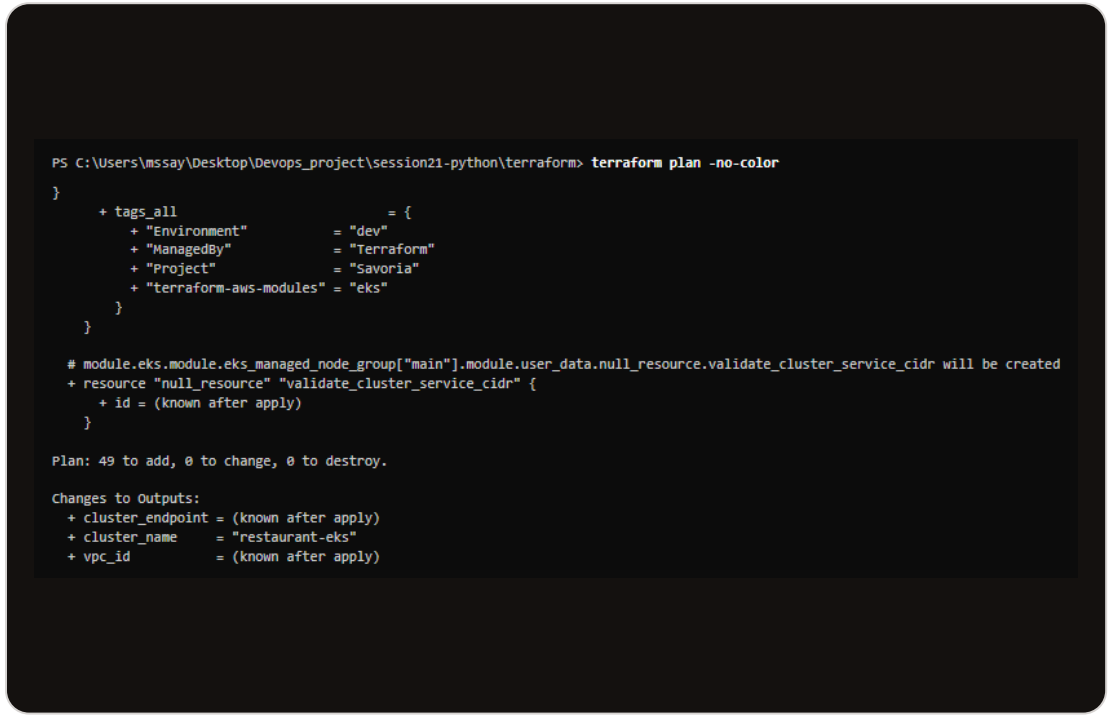
Fulfilling M7 criteria: valid HCL, VPC with 2 public subnets, EKS cluster and terraform.tfvars.example

1. **Two-AZ VPC**
   10.20.0.0/16 VPC with two public subnets in ap-south-1a and 1b and no NAT gateway, to keep cost low.

2. **Managed EKS cluster**
   EKS 1.31 control plane with a managed node group of two t3.small nodes.

3. **Validated plan**
   terraform init and validate succeed; terraform plan reports 49 to add, 0 to change, 0 to destroy.

4. **Secrets-free configuration**
   terraform.tfvars.example holds placeholders; state and tfvars files are git-ignored.



Evidence: terraform plan, 49 resources to add

Helm Chart with Ingress Routing & Autoscaling

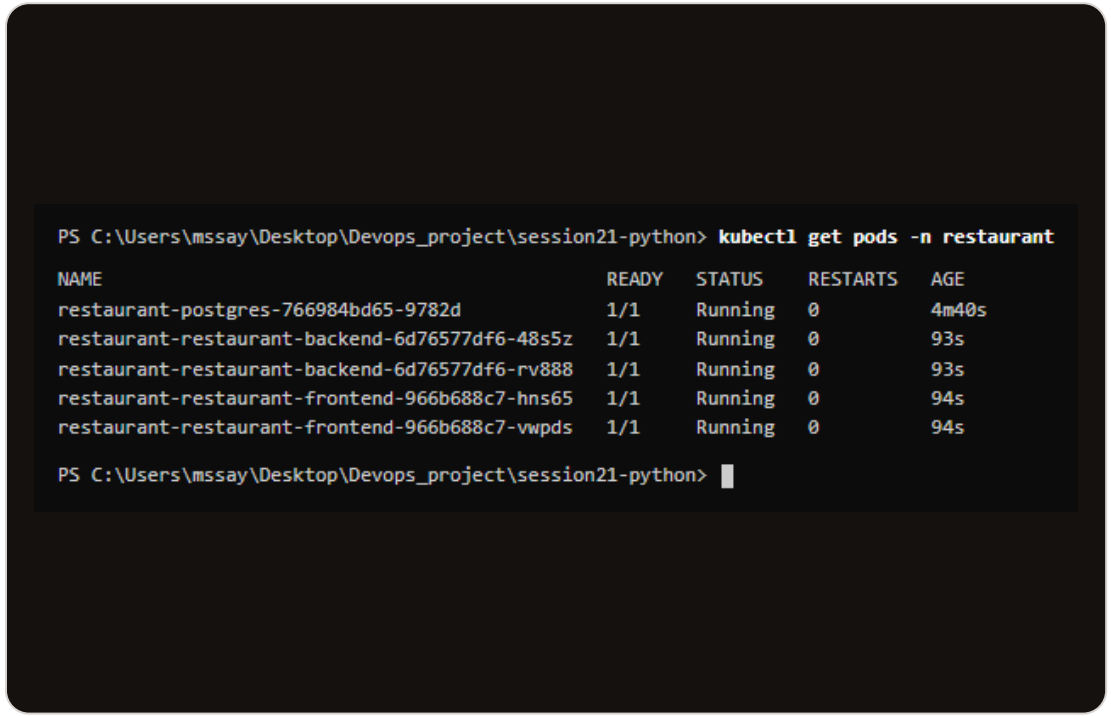
Fulfilling M8 criteria: namespace.yaml, Helm chart, 2+ replicas, ClusterIP services, Ingress and Running pods

1. **Helm chart for the whole app**
   helm/restaurant templates backend, frontend, PostgreSQL, Ingress, HPA and ServiceMonitor.

2. **Namespace isolation**
   k8s/namespace.yaml creates the restaurant namespace that holds every workload.

3. **Replicas and autoscaling**
   Two replicas each for backend and frontend; the HPA scales the backend 2 to 6 pods at 60% CPU.

4. **Ingress host routing**
   NGINX Ingress on restaurant.local sends / to the frontend and /api to the backend (ClusterIP).



Evidence: kubectl get pods -n restaurant, all Running (local kind cluster)

Prometheus Scraping & Grafana Dashboard

Fulfilling M9 criteria: /metrics endpoint, Prometheus target UP and a Grafana dashboard with live panels

1. **Metrics endpoint**
   FastAPI exposes /metrics; a ServiceMonitor is set to scrape it every 15 seconds.

2. **Target verified UP**
   The Prometheus Targets page shows restaurant-backend with 1 of 1 targets up.

3. **Live Grafana panels**
   Request rate by endpoint, memory (RSS), CPU usage, backend target status and total API requests.

4. **Data from real traffic**
   The dashboard showed a HEALTHY (UP) backend and 344 cumulative API requests.



Evidence: Grafana backend and infrastructure overview

Troubleshooting Drills & Live Demo Readiness

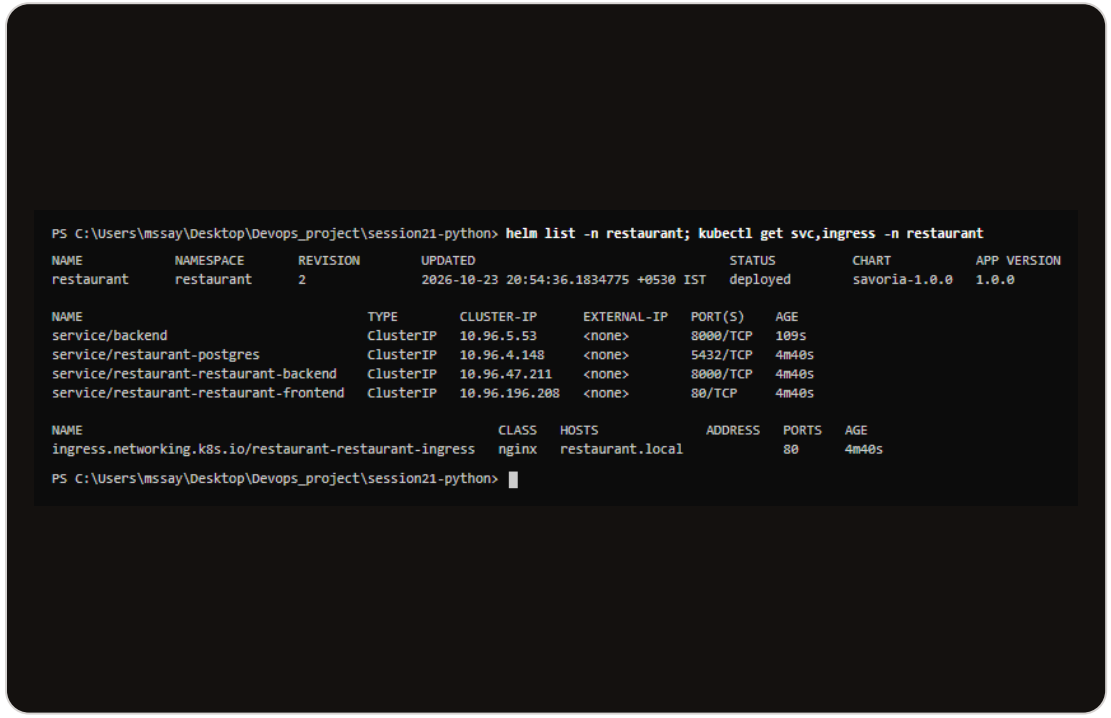
Fulfilling M10 criteria: professional README, failure-scenario exercises and a demo walkthrough

1. **Drill 1: ImagePullBackOff**
   A pod with a non-existent tag; kubectl describe shows the pull error; fix by pinning a valid GHCR SHA tag.

2. **Drill 2: Empty endpoints**
   kubectl get endpoints shows none; the Service selector matched no pods; fix by aligning the labels.

3. **Complete documentation**
   README covers architecture, quickstart, CI/CD, Terraform, Helm and an M1 to M10 evidence table.

4. **End-to-end demo flow**
   Commit, CI tests, Trivy scan, GHCR push, then Helm rolling update on the cluster.



Evidence: helm list and ingress for the restaurant release